第 4 章練習：執行緒

先列出必要前提與中間狀態，再對照獨立的詳解。

OS04-E01　共享範圍

一個行程內有兩條執行緒，各自執行一次 `counter++`；`counter` 是初值為 0 的共享全域整數，兩次遞增均正常完成且不會溢位。

1. 在讀取、加一、寫回的抽象模型中，為何兩次遞增可能只留下 1？若使用 C/C++ 非原子整數且沒有同步，語言層面的性質為何？
2. 將 counter 改成每條執行緒各自的 thread-local 物件，消除了什麼競爭？又失去什麼？
3. 若所有相關存取都使用同一把 mutex，完整遞增放在臨界區，最後結果為何？

OS04-E02　並行不保證加速

固定問題規模。單核心基準執行時間中，80% 可在 8 個核心上理想平分，20% 必須循序執行；先忽略額外開銷。

1. 使用簡化的 Amdahl 模型計算加速上限。
2. 哪些因素可能讓實測加速比較低？
3. 若基準時間中必須循序的比例降為 5%，其餘 95% 可理想平分，新的上限為何？

OS04-E03　取消與清理

Worker 擁有 buffer B、持有 mutex M，且已註冊能釋放兩者的清理處理器。它在 `pthread_cond_wait` 等待網路回覆的條件時，主執行緒要求取消它。

1. 非同步取消（asynchronous cancellation）為何可能使其他工作無法前進或資料不一致？
2. 延後取消（deferred cancellation）如何提供較可控制的清理時機？取消請求成功是否代表執行緒已結束？
3. 對鎖、共用資料和參數生命週期，還需要哪些額外設計？

OS04-E04　M:N 映射與阻塞補償

某 M:N runtime 有 100 條使用者執行緒與固定 10 個核心 carrier。每個任務都要發出一次真正會阻塞 carrier 的系統呼叫；先假設沒有非同步 I/O 或替代 carrier。

1. 分別在 1 個與 10 個 carrier 阻塞後，還有多少 carrier 可執行？其餘尚未發出呼叫的工作在哪裡等待？
2. 為何不能由此推論所有 M:N 系統最多只有 10 條使用者執行緒等待？
3. 比較 many-to-one 與 one-to-one：一條執行緒的阻塞會如何影響其他執行緒？

OS04-E05　建立、join 與 TLS

主執行緒持有 `Work{input=14, output}`，worker 計算 `output=3×input`。另有一個兩條 worker 的執行緒池重複處理不同工作。

1. 主執行緒何時可無衝突地讀出 42？等待 10 ms 足以保證嗎？
2. 分別說明 Pthreads、Windows 與 Java 中成功等待及失敗或中斷的差別。
3. 回傳 worker 堆疊區域變數的指標，或在 detach 後立即釋放其參數，是否安全？
4. 兩條執行緒的 TLS counter 各自從 0 加到 1。如何正確合併為 2？重用的 worker 為何不能把 TLS 當作每個工作都自動清空的狀態？

OS04-E06　Amdahl、DAG 與 false sharing

固定問題規模，單核心基準時間為 100 ms，其中 20 ms 循序、80 ms 可理想平分。另一項工作用四層滿二元樹合計八個葉節點的數值 1 至 8，先將相鄰葉節點配對；每個 leaf 與 merge 各需 1 單位時間。另假設 cache coherence line 為 64 bytes、`int` 為 4 bytes，陣列起點對齊 64 bytes。

1. 忽略開銷，計算 8 個核心的時間、加速比、效率及核心數趨近無限時的加速上限。若再增加總共 10 ms 的啟動與同步開銷，結果如何？
2. 為何 80% 的操作數不一定代表 80% 的基準執行時間？請舉數值反例。
3. 列出 reduction 的中間和、總 work、span 和任意 P 個處理器下的執行時間下界。
4. 不同核心上的 W1 反覆寫入 `a[0]`，W2 反覆寫入同一 coherence line 內的 `a[1]`；若沒有其他衝突，是否一定有 data race？是否仍可能發生 false sharing？

OS04-E07　執行緒池與 serial queue 的等待環

只有兩條 worker 的 pool 同時執行 A、B。A 與 B 各自提交子工作 A1、B1，然後阻塞等待其子工作；此 pool 不提供等待期間協助執行子工作的機制。

1. 為何即使工作本身的相依圖沒有環，整個系統仍可能卡住？
2. 提出避免此問題的設計；不要假設所有 pool 都具備 work-stealing 或 help-join。
3. 一個正在 serial queue Q 執行的 block，同步提交另一個 block 到 Q 並等待它，會發生什麼事？
4. 四條 worker 分別計算 `[1,2]`、`[3,4]`、`[5,6]`、`[7,8]`，如何以 reduction 加總 1 至 8？

OS04-E08　取消、fork 與 signal

Worker 持有 mutex M 和 buffer B，先後註冊 `unlock-M`、`free-B` 清理處理器，然後在 `pthread_cond_wait` 等待條件。另有一個多執行緒行程，其中 T2 持有鎖時 T1 呼叫 `fork`。

1. 取消請求成功是否表示 worker 已退出？
2. 在 condition wait 期間處理延後取消時，mutex 和兩個清理處理器依什麼順序動作？
3. 若取消發生在共享資料結構只更新一半時，單純 unlock 是否一定能恢復正確性？
4. T1 的子行程保留哪些執行緒？說明 fork 後的 signal disposition、呼叫者的 signal mask，以及子行程的 pending signals。

OS04-E09　Windows 與 Linux 固定版本案例

固定 Windows 7：`CreateProcessW` 成功建立一般行程與初始執行緒，之後再成功呼叫一次 `CreateThread`。固定 Linux 3.2：以單一一般使用者 task 為起點，比較成功執行一般 `fork` 與成功執行使用 `CLONE_VM`、`CLONE_FS`、`CLONE_FILES`、`CLONE_SIGHAND`、`CLONE_THREAD` 的同群組 clone；只計原 task 與新 task，排除其他 task。

1. Windows 案例有幾條執行緒與幾個 TEB？哪些資源共用？
2. 分別列出兩種 Linux 建立方式中的 task、mm、page table 與 files 結構數量及共享關係。
3. 若只有 VM／FS／FILES／SIGHAND 等共享旗標並帶 `SIGCHLD`，但沒有 `CLONE_THREAD`，是否必然屬同一 thread group？
4. 共享 mm 是否使未同步的 heap 寫入自動安全？

OS04-E10　排程實體與 CPU 份額

單 CPU 上，P 有 3 條、Q 有 2 條持續可執行的執行緒；所有核心排程實體等優先權，長期 CPU 時間平均。假設 many-to-one 時每個行程各有一個 carrier，one-to-one 時每條執行緒各有一個核心實體。

1. 分別求兩種映射下 P、Q 的 CPU 份額。
2. 為何不能不加條件地推論「多開執行緒就必然得到更多 CPU」？給出一種改變結果的排程政策或工作狀態。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 4. Linux 3.2 fork.c; pthread_join(3); pthread_cond_wait(3p); C++ thread storage; OpenMP reduction.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study